The Relay

What I built and why

Python/asyncio on Groq's free tier: qwen3.8-27b routes, BM25 fetches 3 pieces of 2400 characters, a decide step refuses weak evidence, gpt-oss-120b drafts with citations, and a gate checks every sentence before release. **The decision that shaped it most: an answer ladder with a no-model floor.** If the limiter predicts no room within 15 s, a request steps down (another model, a lean answer, then cited passages with no model call and an estimate of when to retry) instead of queueing. The floor needs no budget: with the daily budget gone it replied in 4 ms at p50 (stand-in).

The real service and my stand-in

Table 1: limits found in the real service

limit	value found	how found	confidence
Requests per minute	30 per model	429 body "Limit 30" after 30 successes in a probe burst; qwen never hit, documented	high (qwen low)
Tokens per minute	8000 per model; qwen also output 1000, input 7000 (undocumented)	x-ratelimit-limit-tokens; qwen's two only in real-run 429 bodies	high
Requests per day	1000 per model	x-ratelimit-limit-requests; refills: 120b remaining 942 to 999 in 3.76 h, same day	high (refill medium)
Tokens per day	200000 per model (documented)	no header; 0 of 22 real 429s named a daily limit	low
Requests in flight at once	none found (20b k=8; 120b k=8; qwen k=4)	stepwise concurrency in the probe	medium (lower bound)
What it returns when a limit is hit	429, retry-after 2 s, body naming the limit ("RPM: Limit 30, Used 30")	probe burst; real-run 429 bodies	high
Time until it recovers	2.0 s to the next admission; tokens refill 133/s (TPM/60)	recovery polls; token headers after a draft	high

From a probe of 171 real calls and real 429 bodies. The stand-in speaks Groq's API, headers and 429 wording with the same buckets, first-byte and streaming rates fitted per model to the probe (gpt-oss-120b: 13 calls, 473 tokens/s), load slowdown, stalls and failures, and replays real replies. **Least like Groq:** it replays only drafts that passed the check, so it never shows a gate failure: on identical arrivals the real steady run gave 87.1% a written answer, its twin 100.0%.

Results

Table 2a: latency per scenario

scenario	source	requests sent	duration (s)	offered rate/min	TTFT p50 / p95 / p99 (ms)	total p50 / p95 / p99 (ms)	n (TTFT / total)
burst	stand-in	34	266	7.7	948 / 7828 / 11641 (= max, n<100)	948 / 7828 / 11641 (= max, n<100)	34 / 34
chaos	stand-in	27	274	5.9	1966 / 9509 / 14496 (= max, n<100)	2013 / 14496 / 17112 (= max, n<100)	26 / 27
daily_exhausted	stand-in	14	261	3.2	4 / 9 / 9 (= max, n<100)	4 / 9 / 9 (= max, n<100)	14 / 14
overload	stand-in	78	299	15.6	1730 / 14043 / 28404 (= max, n<100)	1961 / 16453 / 30882 (= max, n<100)	74 / 78
overload_naive_retry	stand-in	78	294	15.9	6271 / 11547 / 18414 (= max, n<100)	6271 / 11547 / 18414 (= max, n<100)	77 / 78
real_burst	real	20	21	56.1	1006 / 11419 / 11419 (= max, n<100)	1257 / 11419 / 15963 (= max, n<100)	18 / 20
real_burst	stand-in	20	17	72.3	718 / 7117 / 7117 (= max, n<100)	718 / 7117 / 9219 (= max, n<100)	19 / 20
real_steady	real	31	849	2.2	2320 / 4192 / 5549 (= max, n<100)	2320 / 4192 / 5549 (= max, n<100)	31 / 31
real_steady	stand-in	31	850	2.2	1495 / 8248 / 9004 (= max, n<100)	1495 / 8248 / 9004 (= max, n<100)	31 / 31
rpm_halved	stand-in	36	289	7.5	3135 / 12601 / 16205 (= max, n<100)	3522 / 12601 / 16205 (= max, n<100)	34 / 36
steady	stand-in	24	264	5.4	2029 / 9630 / 16001 (= max, n<100)	2029 / 9630 / 16001 (= max, n<100)	24 / 24

Table 2b: outcomes per scenario

scenario	source	succeeded %	of which passages after a failed check %	rate-limited %	timed out or given up %	error %	retries per request	peak queue length	tokens per answer	cost per 1,000 answers (free / shadow)
burst	stand-in	70.6	0.0	29.4	0.0	0.0	0.06	4	1629	$0 / $0.83
chaos	stand-in	96.3	0.0	0.0	3.7	0.0	0.26	2	2916	$0 / $1.43
daily_exhausted	stand-in	0.0	0.0	100.0	0.0	0.0	0.21	1	0	$0 / $0.00
overload	stand-in	87.2	0.0	7.7	5.1	0.0	0.01	5	1483	$0 / $0.51
overload_naive_retry	stand-in	46.2	0.0	52.6	1.3	0.0	5.81	1	1032	$0 / $0.66
real_burst	real	55.0	10.0	35.0	10.0	0.0	0.05	4	1385	$0 / $0.64
real_burst	stand-in	65.0	0.0	30.0	5.0	0.0	0.00	4	1355	$0 / $0.58
real_steady	real	100.0	12.9	0.0	0.0	0.0	0.00	1	3713	$0 / $1.86
real_steady	stand-in	100.0	0.0	0.0	0.0	0.0	0.00	1	3252	$0 / $1.77
rpm_halved	stand-in	86.1	0.0	8.3	5.6	0.0	0.03	3	1352	$0 / $0.69
steady	stand-in	100.0	0.0	0.0	0.0	0.0	0.04	1	3130	$0 / $1.50

Real rows are real runs; the rest are the stand-in; percentiles nearest-rank. **The number we could not have guessed:** qwen3.8-27b also caps output at 1000 tokens a minute (12.5% of its documented TPM) and input at 7000, in no document or header, only in 429 bodies. Quality on 18 questions I wrote (4 unanswerable): 72.2% correct and cited, 22.2% correctly refused, 1948 tokens per answer.

Where the time goes

real_burst-20261111T073012Z-0009 (Q14) took 15963 ms and the person left at 15.9 s, before any token. explain_request.py on its spans: waiting 15122 ms, working 838, retrying 0. It waited 5625 ms for gpt-oss-120b, drafted in 838 ms, failed the citation check, then waited 9496 ms for a second draft on gpt-oss-20b: in effect a retry, and the person got nothing.

When a limit is hit

Per minute (stand-in overload): 87.2% written (61.5% from a fallback model or the lean step, the weaker answers), 7.7% passages, 5.1% gave up; others are delayed up to the ladder's limit (queue wait p95 10572 ms); nobody is turned away. **Per day:** 0.0% written, 100.0% passages: straight to the floor rather than holding people until a reset.

Retrieval: quality versus cost

Table 3: retrieval settings

setting	questions asked	correct and cited %	wrong %	correctly refused %	ungraded %	unsure	degraded	tokens per answer	TTFT p50 / p95 (ms)	total p50 / p95 (ms)	budget wait p50 (ms)
large	18	72.2	5.6	22.2	0.0	0	1	1948	4622 / 16558	4622 / 16558	3552
medium	18	50.0	27.8	22.2	0.0	0	2	2212	12234 / 29223	12234 / 29223	11130
medium+rerank	18	50.0	27.8	22.2	0.0	0	1	1769	8882 / 23195	8882 / 23195	7659

small_lean	18	33.3	44.4	22.2	0.0	7	1	1035	3579 / 19596	3579 / 19596	2697
small	18	27.8	50.0	22.2	0.0	8	2	1957	11146 / 26696	11146 / 26696	10010

large: most correct at 1948 tokens per answer; re-ranking saved tokens without adding correct answers; small_lean is cheapest but 33.3% correct. Near the limit the lean step fetches 2 pieces and skips the router.

How this library will mislead a naive retriever

Plain BM25, raw question (data/retrieval/traps.md):

- **Different words:** "How does H-Net decide where to cut the byte stream into pieces?" shares 0 of 7 terms with its answer; 0 of 5 settings got it right.
- **Text about a table outranks it:** for "What training cost in FLOPs does Table 2 report for the big Transformer?" the table is rank 2.
- **Unanswerable but plausible:** "What is H-Net's p95 latency serving 1,000 simultaneous requests?" tops with moshi on "latency"; the decide gate refused it in 5 of 5 settings.

Retries

Ships: a failed call (429, 5xx, timeout) moves to the next model in the fallback order, each at most once per step, then down the ladder; no resend to the same model. **Rejected:** 1 retry per step within 10% of recent calls left 13 and 14 people without a written answer against 12 and 10 (stand-in, two seeds). **Worse:** a naive client (3 same-model retries, no limiter; stand-in) left 25 and 23 of 47 without one against 0 and 0, drawing 274 and 264 rejections.

What the person sees

Daily budget gone; "What theoretical and practical latency does the paper report?" gets, after 4 ms (stand-in):

> No written answer this time: here are the most relevant passages from the papers, quoted as they are. A written answer should be possible in about 496 s.
> [S1] 2410.00037v1 pp.25-26 §5.3: "Table 5 reports results for ablations on the use of an RQ- Transformer. We first replicate the setting of Copet et al. (2023) with the delay pattern of [0, 1, 2, 3, 4, 5, 6, 7], which means that each level of R VQ token is generated one timestep after the preceding level. In this context, we see that using an RQ-Transformer is not necessary, as it only provides a marginal improvement in perplexity. However, this delay pattern induces a theoretical latency of 8 timesteps, which amounts to 640ms, a latency that is incompatible with the requirements of a real-time dialogue model."
> [S2] 2410.00037v1 p.26 §5.3: "Results - Additional ablations. Table 6 reports additional ablations on additional delay patterns, the weight of the semantic token loss and our proposed Inner Monologue procedure, all using the RQ-Transformer. First, we compare three configurations of delays that are compatible with real-time dialogue. The [0 , 0, 0, 0, 0, 0, 0, 0] pattern represents the minimal latency of 80ms that can be obtained with Mimi tokens at 12.5Hz."

Honest and immediate but, without the router, the passages miss the figures asked for (graded wrong).

What I'm unsure about, what I left out, what I learned

Least certain: Groq's daily token limit, never shown in a header or 429. Evidence says it refills: our key was charged 263495 gpt-oss-120b tokens in one rolling 24 h window, over the documented 200000, with no daily 429, and the request counter refills (Table 1). If it resets at a fixed hour instead, the estimate above is wrong by hours. **Not built:** sentence-by-sentence release (we buffer, so first token equals total); a model-based grader (it spends shared budget). **Next:** a router prompt for Q14; fetching the neighbouring piece when a draft's number is missing (Q2).

One new thing, in my own words: One thing I learned is that a response starting quickly does not mean the system is fast. Time to first token is how long the person waits before seeing anything; total time is how long until the whole answer has arrived. At the model the two differ: the first piece arrives quickly and the rest streams after it. In my pipeline they are the same, because every reply is checked before it is released, so the person sees nothing until the checked answer is ready. That moved the question from "how fast does the model start" to "where does the waiting happen". Splitting one slow request into waiting, working and retrying showed it spent almost all its time waiting for budget, not generating. An average would have hidden that; percentiles and a per-request split did not.

Honesty notes. Tables from runs measured at e7aa65f; 11 rows later moved from rate-limited to succeeded (passages after failed checks). The fixed gate passes 127 of 165 saved real drafts instead of 119, changing 2 requests in the tables; the replay is its measurement (one real rerun would be anecdotal). The grader changed after I saw answers (results/quality/grade_changes_*.md) and a Q13 gold fact became optional; 37 of 177 graded real rows were never checked by hand. To avoid waiting, two eval jobs capped at the documented limit, and the real load runs and a last Q14 check used refill accounting. Runs on a second key (4, one budget) and a mistyped key (2, 401s) are excluded.

Your feedback on Vaani

What I liked about Vaani was that it brings different parts of building a voice agent together in one platform. Instead of focusing only on speech recognition or voice generation, it also covers agent setup, tool integrations and call analysis. I think this makes it useful for developers looking to build practical voice-based applications.

One thing I found a little confusing was the latency information across the website. Different pages mention different response times, but it was not very clear what exactly was being measured or how the numbers compare.

One change I would suggest is adding a simple end-to-end example showing how someone can create an agent, connect it to tools, make a call and analyse the result. This would make the platform easier to understand, especially for someone exploring it for the first time.

How I worked with the assistant

My prompts from the main working session are attached verbatim (prompts.md).

Three prompts that mattered. First, the ground rules I set before any code: measure before designing, never make a person wait for budget that isn't coming, and every reported number must come from a file a script produced. They became the ledger with its caps, the answer ladder with a no-model floor, and the number checker behind this page. Second, "before any load testing, measure answer quality on real groq": quality became the first measurement, so the retrieval setting and the drafter were chosen on graded answers, not on speed. Third, the gate fix: "dont accept uncited sentences", and replay every saved real draft through the old and new gate before any real call. That turned a change I could only have shown with one anecdotal rerun into a measurement over every draft we had.

What it got wrong, and how I caught it. While grading, the assistant said a retrieved passage did not contain "discarding the prediction of the model for them" and treated an answer as unsupported. I rebuilt the pieces, found the phrase in the source, and asked it to recheck how it had decided; the answer stayed correct. I caught it by reading the source instead of the assistant's summary of it.

Where I overruled it. It proposed letting the citation gate accept an uncited opening or closing sentence. I refused: no sentence reaches a person without a citation; filler can be dropped, anything else fails the draft. Earlier it suggested counting the daily budget as a refilling bucket; I kept the conservative rolling sum until there was evidence, and when I later used refill accounting to avoid waiting, the honesty notes say so.

What is distinctly mine. The ground rules; the questions and their gold facts; the grading decisions and overrides; going back to one API key after re-reading the brief; and the call to stop spending budget once the replay had measured the fix.